

Topic 9: Binary Search Trees

9.1 BST Property

- For every node N: all values in the left subtree $< N.data$; all values in the right subtree $> N.data$
- This ordering property enables $O(\log n)$ search, insert, and delete for balanced trees
- Inorder traversal of a BST always yields elements in sorted order

9.2 BST Search and Insert

- Search: compare target with root -- go left if smaller, right if larger, stop if equal or null
- Insert: perform search until nullptr is reached, then create a new node at that position
- Both $O(\log n)$ average, $O(n)$ worst case (skewed tree)

```
TreeNode* insert(TreeNode* root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->data) root->left = insert(root->left, val);
    else if (val > root->data) root->right = insert(root->right, val);
    return root;
}

TreeNode* search(TreeNode* root, int val) {
    if (!root || root->data == val) return root;
    if (val < root->data) return search(root->left, val);
    return search(root->right, val);
}
```

9.3 BST Deletion

- Case 1: Leaf node -- simply delete it
- Case 2: Node with one child -- replace node with its child
- Case 3: Node with two children -- replace data with inorder successor (smallest in right subtree), then delete the successor

```
TreeNode* minNode(TreeNode* n) {
    while (n->left) n = n->left;
    return n;
}

TreeNode* deleteNode(TreeNode* root, int val) {
    if (!root) return root;
    if (val < root->data) root->left = deleteNode(root->left, val);
    else if (val > root->data) root->right = deleteNode(root->right, val);
    else {
        if (!root->left) { TreeNode* t = root->right; delete root; return t; }
        if (!root->right) { TreeNode* t = root->left; delete root; return t; }
        TreeNode* succ = minNode(root->right);
        root->data = succ->data;
        root->right = deleteNode(root->right, succ->data);
    }
    return root;
}
```